

Diabetes Severity Prediction Using Decision Tree Classification on a Synthetic Clinical Dataset

Table of Contents

1. Introduction & Objective

2. Synthetic Diabetes Dataset (Scikit-learn – Used for This Project)

2.1 Exploratory Analysis

3. Data Preprocessing & Splitting

3.1 Data Preprocessing

3.2 Data Splitting

3.3 Summary

4. Model Implementation & Training

4.1 Model Setup

4.2 Training Process

4.3 Hyperparameter Tuning and Justification

5. Results & Evaluation

5.1 Interpretation of Results

5.2 Comparison with Other Configurations

5.3 Conclusion on Model Performance

6. Discussion: Underfitting vs Overfitting

7. Deployment Demo (Gradio Application)

8. Conclusion

9. Appendix: Python Code

1. Introduction & Objective

Diabetes is a chronic metabolic disease that can lead to severe health complications if not properly managed. Predicting disease progression or severity using clinical indicators is an important application of machine learning in healthcare. The primary objective of this project is to build a machine learning model capable of predicting **diabetes severity level** using patient health measurements. This problem is formulated as a **binary classification task**, where the target variable indicates whether a patient's diabetes severity is classified as *high* (1) or *low* (0).

The chosen algorithm for this task is the **Decision Tree classifier**, which is interpretable, non-parametric, and well-suited for tabular clinical data. In alignment with the original assignment requirements, this project uses the **synthetic diabetes dataset provided by Scikit-learn**, which is widely adopted for educational and benchmarking purposes. Although the dataset is synthetic and does not represent raw clinical records, it provides realistic feature relationships that allow meaningful experimentation with model training, evaluation, and hyperparameter tuning. The focus of this project is therefore on **model behavior, generalization, and deployment**, rather than medical diagnosis.

2. Synthetic Diabetes Dataset (Scikit-learn – Used for This Project)

This project uses the **Scikit-learn diabetes dataset**, which contains **442 patient samples** and **10 normalized clinical features**. Unlike diagnostic datasets, this dataset is designed to model **diabetes disease progression**, which we adapt into a **binary severity classification task**.

Features

The dataset includes the following input features:

- age – age (normalized)
- sex – sex (normalized)
- bmi – body mass index
- bp – average blood pressure
- s1 – total serum cholesterol
- s2 – low-density lipoproteins
- s3 – high-density lipoproteins
- s4 – total cholesterol / HDL
- s5 – triglycerides
- s6 – blood sugar level

The original continuous target variable was **binarized** using a threshold to represent **lower severity (0)** and **higher severity (1)** diabetes progression.

Exploratory Analysis

A brief exploration of the dataset shows:

- Class distribution:** The binarized target produces a reasonably balanced dataset, suitable for binary classification without heavy resampling.
- Feature scaling:** All features are already **normalized**, removing the need for additional scaling.
- Data quality:** No missing values are present, and all features are numeric.

Because the dataset is synthetic and standardized, it allows a clean focus on **algorithm behavior and hyperparameter effects**, without the noise introduced by real-world data inconsistencies.

3. Data Preprocessing & Splitting

3.1 Data Preprocessing

- **Target transformation:** The original continuous target variable was converted into a **binary severity label** using a threshold, transforming the task into classification.
- **Feature selection:** All 10 available clinical features were retained as input variables (X).
- **Missing values:** No missing values were present, so no imputation was required.
- **Normalization:** No additional normalization was applied, as features are already standardized.

3.2 Data Splitting

- The dataset was split into **training and testing sets** using an **80/20 split**.
- Stratified sampling was applied to preserve class balance.
- A fixed `random_state=42` ensures reproducibility.

3.3 Summary

This preprocessing pipeline ensures a clean, leakage-free dataset and allows reliable evaluation of the Decision Tree classifier’s generalization performance.

4. Model Implementation & Training

The **Decision Tree classifier** was selected due to its interpretability and effectiveness for tabular data. It recursively partitions the feature space to maximize class purity, making it particularly suitable for analyzing structured clinical datasets.

4.1 Model Setup

The model was configured using the following hyperparameters:

- `max_depth = 5`
- `min_samples_split = 10`
- `min_samples_leaf = 5`

These values were chosen through controlled experimentation to balance bias and variance.

4.2 Training Process

- The model was trained using only the **training set (80%)**.
- The **Gini impurity** criterion was used to determine optimal splits.
- No feature scaling or encoding was required.

4.3 Hyperparameter Tuning and Justification

max_depth	min_samples_split	min_samples_leaf	Train Acc	Test Acc	F1 Score	Observation
1	50	20	0.76	0.69	0.39	Underfitting
2	1	1	1.00	0.67	0.46	Overfitting
5	10	5	0.80	0.77	0.70	Optimal
5	5	5	0.80	0.77	0.70	Stable

Justification: The selected configuration provides the **best trade-off between recall, precision, and generalization**, achieving strong test performance without overfitting.

5. Results & Evaluation

Metric	Value
Train Accuracy	0.801
Test Accuracy	0.773
Precision	0.651

Recall Metric	0.759 Value
F1 Score	0.701

5.1 Interpretation of Results

- **Accuracy:** Indicates strong overall predictive performance.
- **Recall:** High recall ensures most severe cases are correctly identified.
- **F1 Score:** Confirms a balanced precision–recall tradeoff.

5.2 Comparison with Other Configurations

- Shallow trees underfit and miss severe cases.
- Deep trees overfit and generalize poorly.
- The final model maintains stability and robustness.

5.3 Conclusion on Model Performance

The Decision Tree demonstrates **consistent generalization** and meaningful predictive ability on synthetic diabetes severity data.

6. Discussion: Underfitting vs Overfitting

The experiments clearly illustrate the **bias–variance tradeoff**:

- Underfitting arises from overly constrained trees.
- Overfitting occurs with deep, unconstrained trees.
- Controlled depth and minimum samples produce the best results.

7. Deployment Demo (Gradio Application)

A **Gradio-based web interface** was implemented:

- Users input normalized clinical features.
- The trained Decision Tree predicts **diabetes severity**.
- The model is loaded from a saved `.pkl` file.
- No training logic is exposed to the user.

This demonstrates **real-world deployment readiness**.

8. Conclusion

This project demonstrates a complete machine learning pipeline using a **Decision Tree classifier** on a **synthetic diabetes severity dataset**:

- Clean preprocessing and splitting
- Hyperparameter tuning
- Robust evaluation
- Interactive deployment

The project emphasizes **model interpretability, generalization, and deployment**, aligning well with academic and practical expectations.

9. Appendix: Python Code

```
- 01_data_exploration.ipynb
- 02_model_training.ipynb
- train_model.py
- diabetes_gradio_app.py
```